Sentiment Analysis of Movie Reviews: Can a Model Understand "Not"?

CSC 425 AI Term Project, Milestone 2: Data Preparation & Baseline Prototype
Obaid Alnaqbi (solo project)

1. Introduction

Sentiment analysis is the ability to read a text such as a film review and determine its sentiment, whether it is positive or negative. I do it myself before going to see a film, and companies do it on a much greater scale, so they need systems that do this automatically for thousands of reviews.

In my proposal, I decided to create a standard sentiment classifier and, if time permits, a neural network or DistilBERT. Dr. Zhou gave some feedback, and I changed the plan in two ways after it. First, since sentiment analysis is a very well-researched topic, I added one more specific question to make the project my own: is it possible for a model to detect the effect of the word "not" on the meaning of a review? "The movie was not enjoyable" and "The movie was enjoyable" differ by one word but have opposite meanings. Second, I decided not to use the neural network for the time being, and concentrated my efforts on making a simple TF-IDF model and understanding it in depth. I also did not remove the stop words, since the word "not" is in most stop-word lists, and removing them would delete exactly what I want to study.

This milestone demonstrates the basic pipeline working: review text, cleaning, TF-IDF features, a classifier, and a positive/negative label with a confidence score. I experimented with two classifiers and two feature settings on a held-out set of validation reviews and on a negation challenge set that I created.

2. Dataset and Environment

The Large Movie Review Dataset (Maas et al., 2011) from the Stanford AI Lab is used. It contains 50,000 movie reviews labeled positive or negative, where 25,000 are used for training and 25,000 for testing, and each split is exactly half positive and half negative. The labels come from ratings out of 10: 4 or below is negative and 7 or above is positive, so there are no "neutral" reviews. I tried the Stanford download server but it did not respond, so my code downloads the same official splits from the Hugging Face mirror (stanfordnlp/imdb).

Following the instructor's suggestion, I didn't use all 25,000 training reviews yet. I used seed 42 to randomly select 2,000 positive and 2,000 negative reviews, and then split them 80/20 into training and validation sets (3,200/800), stratified by label. Models are compared using the validation set during development. The official test set of 25,000 reviews is not used at all in this milestone. I will only touch it once, at the end, after I have chosen my final approach.

Split	Reviews	Positive	Negative	Mean words	Median words	Max words	Contain "not"
train	3200	1600	1600	231.3	172	1501	59%
validation	800	400	400	230.6	175	1808	59%

Table 1. Statistics for the subset used in this milestone (after cleaning).

The classes are perfectly balanced, so accuracy is a reasonable measurement and I don't need class weighting. The lengths of reviews vary a lot: 100 to 300 words is typical, but some reviews

exceed 1,500 words (Figure 1). The length distributions of positive and negative reviews are approximately the same, so length alone would not help the model. One number really leaped off the page: 59% of the reviews include the word "not". Negation is everywhere in actual reviews, which is another reason not to get rid of it.

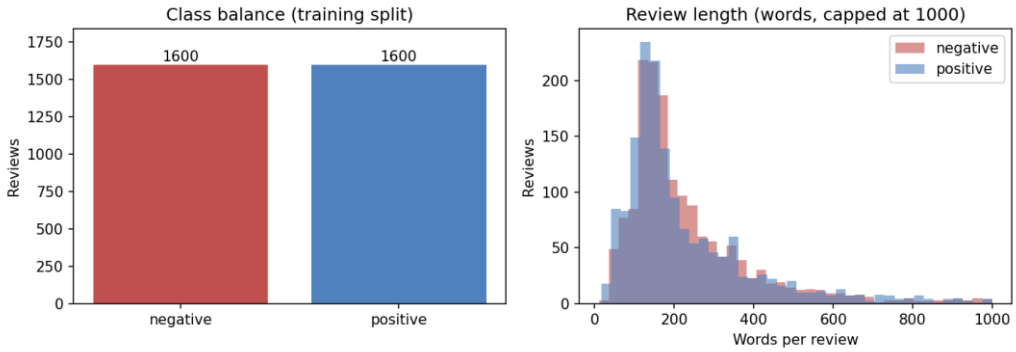


Figure 1. Class balance and review length in the 3,200-review training split.

Environment: Python 3.13, scikit-learn 1.7, pandas 2.3, matplotlib 3.10, laptop CPU. All models are trained in less than 1.5 seconds.

3. Preprocessing

I read a number of reviews from each class before deciding on preprocessing. The main noise is the HTML line breaks ("

") in lots of reviews. My cleaning is intentionally light: replace
 tags with spaces, lowercase everything, and collapse repeated whitespace. I do not remove stop words, because doing so would turn "not good" into "good".

Tokenization is performed inside scikit-learn's TfidfVectorizer, which splits text into words of two or more letters and removes punctuation. I found one side effect: a contraction such as "didn't" becomes the token "didn", which is a different feature from "not". My challenge set uses the full word "not", so the test is clean, but this is something I will handle in Milestone 3. Finally, terms that appear in only one review are ignored (min_df = 2), which eliminates typos and very rare names. The dataset does not contain any missing values.

4. Baseline Methodology

TF-IDF (term frequency times inverse document frequency) converts each review into a vector with one number for each vocabulary term. A term gets a high value if it appears in the review but is rare across all reviews, so words such as "the" get low weights, while words such as "masterpiece" get high weights. I use sublinear term frequency (1 + log tf), so a word repeated ten times does not count ten times as much. On top of these features, I train two classifiers with scikit-learn's default parameters:

- **Logistic Regression:** learns one weight per feature; the weighted sum goes through a sigmoid to give P(positive). Positive weights push the prediction toward positive.
- **Multinomial Naive Bayes:** compares how likely each term is in positive vs. negative reviews, under the assumption that terms are independent. A traditional, fast text baseline.

A majority-class predictor, which always predicts the most common label (50% accuracy here), is included as a floor. To test the negation question, I compare two versions of the features with all other things the same (same split, same settings, same seed):

- **Version A:** ngram_range = (1, 1), only single words. "not good" breaks down into two separate features, "not" and "good".
- **Version B:** ngram_range = (1, 2), words plus pairs of adjacent words. "not good" also becomes its own feature, so the model can learn that this pair has a negative association.

Negation challenge set. I created 10 pairs of sentences where adding "not" flips the opinion, such as "I would recommend this movie." (positive) and "I would not recommend this movie." (negative). Eight pairs go from positive to negative. I wanted to make sure that the model was not just treating "not" as a negative word, so two pairs go the other way, such as "The sequel was bad." vs "The sequel was not bad.". I wrote the sentences before training any model and did not change them after seeing the predictions. They only serve for analysis, not for training. Evaluation metrics are accuracy, precision, recall and F1 (positive class) on the validation set, plus the number of challenge sentences and pairs predicted correctly.

5. Initial Results

Model	Features	# features	Accuracy	Precision	Recall	F1
Majority class	-	-	0.500	0.000	0.000	0.000
Logistic Regression	A: words (1,1)	17,291	0.856	0.864	0.845	0.855
Logistic Regression	**B: words+pairs (1,2)**	**82,919**	**0.882**	**0.892**	**0.870**	**0.881**
Naive Bayes	A: words (1,1)	17,291	0.838	0.917	0.742	0.820
Naive Bayes	B: words+pairs (1,2)	82,919	0.862	0.934	0.780	0.850

Table 2. Validation results (800 reviews). Best model in bold.

The pipeline works end to end and all models are well above the 50% floor. Logistic Regression with word pairs (Version B) is the best model, with an accuracy of 88.2% and an F1 of 0.881. For both classifiers, adding word pairs increased the F1 score (+2.6 points for Logistic Regression and +3.0 for Naive Bayes), at the cost of almost five times as many features (82,919 with word pairs vs 17,291 with single words). Naive Bayes has the highest precision (0.93) but much lower recall (0.74 to 0.78). The confusion matrices (Figure 2) show why: Naive Bayes incorrectly classifies a lot of positive reviews as negative (103 for Version A, whereas Logistic Regression misses only 62). Logistic Regression's errors are spread fairly evenly between the two classes.

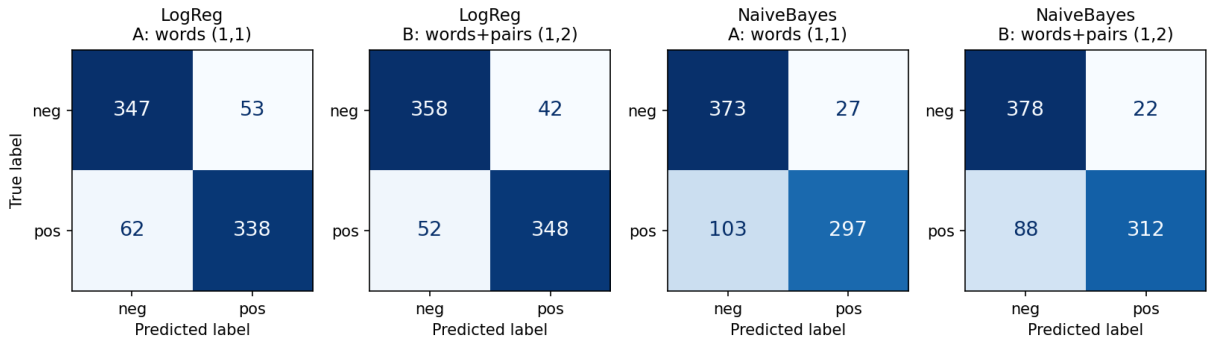


Figure 2. Confusion matrices on the validation set (rows = true label, columns = prediction).

Model	Features	Sentences correct	Original correct	Negated correct	Pairs fully correct
Logistic Regression	A: words (1,1)	13/20	8/10	5/10	3/10
Logistic Regression	B: words+pairs (1,2)	13/20	7/10	6/10	3/10
Naive Bayes	A: words (1,1)	12/20	6/10	6/10	2/10
Naive Bayes	B: words+pairs (1,2)	13/20	6/10	7/10	3/10

Table 3. Results on the negation challenge set (10 pairs, 20 sentences). A pair is only fully correct if both sentences are correct.

The challenge set tells a very different story. The models are around 86 to 88% accurate on real reviews, but get only 12 or 13 out of 20 of my short negation sentences right, and at most 3 out of 10 pairs fully correct. Word pairs made the negated sentences slightly better (Logistic Regression 5 to 6, Naive Bayes 6 to 7), but the number of pairs the model got fully correct did not improve. Table 4 shows the predictions of the best model.

Original sentence	Prediction	Negated sentence	Prediction
The movie was enjoyable.	**negative (0.53) ***	The movie was not enjoyable.	negative (0.60)
I would recommend this movie.	positive (0.55)	I would not recommend this movie.	negative (0.57)
The acting was good.	**negative (0.64) ***	The acting was not good.	negative (0.76)
This film is worth watching.	positive (0.65)	This film is not worth watching.	**positive (0.50) ***
I liked the ending.	positive (0.62)	I did not like the ending.	negative (0.51)
The plot made sense.	**negative (0.64) ***	The plot did not make sense.	negative (0.69)
It was a great movie.	positive (0.86)	It was not a great movie.	**positive (0.72) ***
The jokes were funny.	positive (0.54)	The jokes were not funny.	negative (0.55)
The film was boring.	negative (0.64)	The film was not boring.	**negative (0.76) ***
The sequel was bad.	negative (0.69)	The sequel was not bad.	**negative (0.71) ***

*Table 4. Best model (Logistic Regression, words + pairs) on each challenge pair, with confidence. * = wrong. Pairs 9 and 10 should change from negative to positive.*

Why do the models get these wrong?

Since Logistic Regression assigns one weight to each feature, I could look up the weights of the words in the challenge sentences (Version B model) and explain each mistake:

- **"Not" is treated as a negative word, not as a flip.** The word "not" by itself has a weight of −1.1, because it appears more in complaints. So in "The film was not boring" and "The sequel was not bad", adding "not" makes the prediction more negative than for "The film was boring" and "The sequel was bad" instead of flipping it. The pair "not boring" never appears in the 3,200 training reviews, and "not bad" appears in only 16, with a weight of almost zero.
- **Rare pairs cannot beat strong single words.** The word "great" has a weight of +3.2, while the pair "not great" appears in only 9 training reviews and ended up slightly positive (+0.17). One rare pair is not enough to outweigh one of the strongest positive words, so "It was not a great movie" is predicted positive with 72 to 89% confidence.
- **Some words mean something different on IMDb.** "good" has a weight of only +0.01, almost neutral, probably because reviewers often write "not good", "good actors but..." or "could have been good". "plot" (−1.5) and "acting" (−1.1) are both negative, because people mostly talk about the plot and acting when they are complaining. So "The acting was good" and "The plot made sense" are predicted negative even without any negation.
- **Short sentences are unlike the training data.** Real reviews average 231 words, while mine have 4 to 7 words. With so few features, most confidences are near 0.5 to 0.6, and one or two topic words decide the label.

I also tried Dr. Zhou's two examples in the demo (src/predict.py). "I expected to hate it, but I loved it." is correctly predicted positive (0.75), because "loved" (+1.2) easily outweighs "hate" (−0.2). "The movie was good, but the ending ruined it." is predicted negative, which I would also call correct, but with low confidence (0.55 to 0.59). The demo prints the label and confidence for any review that is typed in.

Summary and next steps

Everything in the baseline is working: data is loaded and cleaned, four models are trained and evaluated, and the best one reaches 88% accuracy on the validation set. What is interesting is that good accuracy on real reviews does not mean the model understands negation. Word pairs work well on real reviews, but they don't do as well on my challenge set, because most "not + word" combinations are rare in 3,200 reviews. In Milestone 3 I will: (1) tune C and alpha on the validation set, (2) add simple negation handling, such as marking the words after "not" ("not good" becomes "not NOT_good") and splitting contractions such as "didn't" into "did not", (3) train on more of the 25,000 training reviews to see if rare pairs become common enough to learn, (4) add mixed-opinion and sarcastic examples to the error analysis, and (5) test the final model once on the official test set.

References

[1] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. Proceedings of ACL 2011, 142-150.

[2] Stanford AI Lab. Large Movie Review Dataset. https://ai.stanford.edu/~amaas/data/sentiment/ (downloaded via the Hugging Face mirror https://huggingface.co/datasets/stanfordnlp/imdb).

[3] Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. Proceedings of EMNLP 2002, 79-86.

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[5] Scikit-learn documentation. Working with text data. https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html